

Exercício 1: Crie uma classe chamada **Funcionario**, que será a superclasse do sistema. Essa classe deve possuir os seguintes atributos:

- string nome
- double salarioBase

Implemente:

- um construtor
- um método calcularSalario()
- um método exibirDados()

Crie uma subclasse:

Gerente:

- possui um setor (string)
- possui um bônus de R\$ 1000,00 (double)
- deve sobrescrever o método calcularSalario()
- deve sobrescrever o método exibirDados()

Regras:

- O método calcularSalario() da superclasse retorna apenas o salário base
- A subclasse deve somar o bônus ao salário base
- Utilize herança para evitar repetição de código
- Os atributos e métodos (quando necessário) devem respeitar o conceito de encapsulamento

Classe Principal:

- crie um objeto do tipo Gerente
- exiba os dados de gerente
- Mostre o salário calculado.
- Inclua comentários explicando o comportamento do programa.

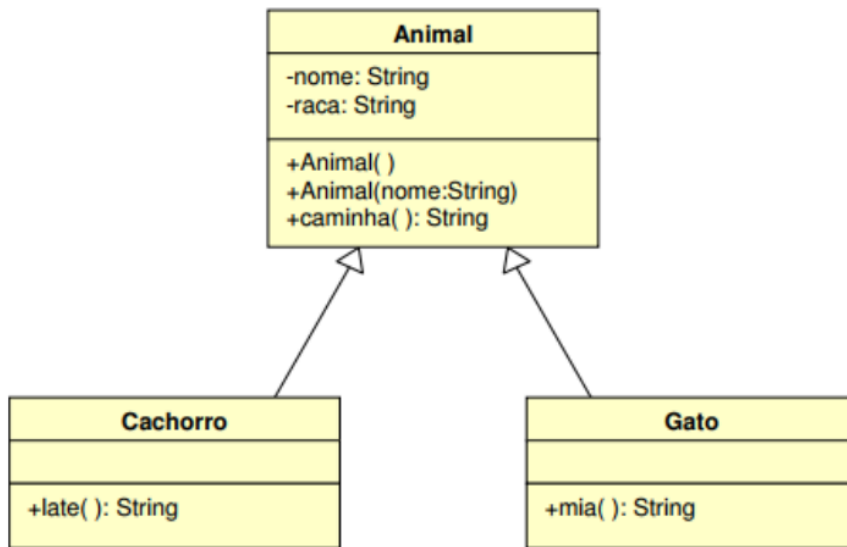
Exercício 2: Crie a classe **Imovel**, que possui um endereço e um preço.

a. crie uma classe **Novo**, que herda Imovel e possui um adicional no preço. Crie métodos de acesso e impressão deste valor adicional.

b. crie uma classe **Velho**, que herda Imovel e possui um desconto no preço. Crie métodos de acesso e impressão para este desconto.

No método de execução, crie um imóvel. Peça para o usuário digitar 1 para novo e 2 para velho. Conforme a definição do usuário, imprima o valor final do imóvel.

Exercício 3: Implemente o diagrama de classes abaixo:



No método main: crie um animal do tipo cachorro e faça-o latir. Crie um gato e faça-o miar. Faça os dois animais caminharem.

Exercício 4: Crie uma classe chamada **Ingresso** que possui um valor em reais e um método `imprimeValor()`.

- crie uma classe **VIP**, que herda **Ingresso** e possui um valor adicional. Crie um método que retorne o valor do ingresso VIP (com o adicional incluído).
- crie uma classe **Normal**, que herda **Ingresso** e possui um método que imprime: "Ingresso Normal".
- crie uma classe **CamaroteInferior** (que possui a localização do ingresso e métodos para acessar e imprimir esta localização) e uma classe **CamaroteSuperior**, que é mais cara (possui valor adicional). Esta última possui um método para retornar o valor do ingresso. Ambas as classes herdam a classe **VIP**

No método main: crie um ingresso. Peça para o usuário digitar 1 para normal e 2 para VIP. Conforme a escolha do usuário, diga se o ingresso é do tipo normal ou VIP. Se for VIP, peça para ele digitar 1 para camarote superior e 2 para camarote inferior. Conforme a escolha do usuário, diga se que o VIP é camarote superior ou inferior. Imprima o valor do ingresso.

Exercício 5: Implemente uma classe **Pessoa** que possui como atributo nome, endereço, e o método `exibirDados()` que retorna uma string.

Crie uma subclasse chamada **Fisica** que herda de **Pessoa**, possui como atributos `cpf` e `dataNascimento`, e os métodos: `exibirDados()` que retorna string e `calcularIdade()` que retorna int.

No mesmo nível da hierarquia, crie uma classe chamada **Juridica**, que estende a **Pessoa**, acrescentando os atributos `cnpj` e `razão social`, e como métodos, `exibirDados()` que retorna string.

No método main, crie instâncias das três classes e analise o comportamento do método `exibirDados()`.

Exercício 6: Modifique o exercício 5 para que o atributo "endereço" da classe **Pessoa** seja do tipo "Endereco".

Exercício 7: Crie uma classe base chamada **Publicacao**:

- `id`: int
 - `titulo`: string
 - `autor`: string
- Método:
- + `exibirFicha()` que retorna uma string com título e o autor.

Subclasse **Livro** que herda de **Publicacao**:

- `isbn`: string
 - `numeroPaginas`: int
- Métodos:
- + `exibirFicha()` sobrescrever a hierarquia acima, concatenando ISBN e número de páginas.

Subclasse **LivroFisico** que herda de **Livro**:

- `localizacaoEstante`: string
 - `pesoGramas`: int
- Métodos:
- + `calcularFrete(double distanciaKm)` que retorna o frete baseado no peso ($\text{peso} * \text{km} * 0,05$)
 - + `exibirFicha()` que sobrescrever a hierarquia acima, adicionando localização e peso

Subclasse **Ebook** que herda de **Livro**:

- `tamanhoMB`: double
 - `formatoArquivo`: string
- Métodos:
- + `gerarLinkDownload()` que retorna uma string simulando um link (ex.: `https://biblioteca.com/download/titulo`)
 - + `exibirFicha()` que sobrescreva o método da hierarquia acima, adicionando o tamanho e formato.

Exercício 8: Crie uma Classe Base **Mensagem**:

- destinatario: string
 - conteudo: string
- Métodos:
- + enviar() que imprime: "Enviando mensagem genérica para [destinatario]".

Subclasse **Email** que herda Mensagem:

- assunto: string
- Métodos:
- Sobrescrever enviar(): imprime "Enviando E-mail para [destinatario] com o assunto [assunto]".

Subclasse **SMS** que estende Mensagem:

sobrescreve enviar() que verifica se o conteúdo tem mais de 160 caracteres. Se tiver, imprime: "Erro: SMS muito longo". Se não, imprime "Enviando SMS para [destinatário]".

Exercício 9: Implemente as seguintes classes:

Classe Usuario

Atributos:

- nome
- cpf

Métodos:

- construtor
- getters e setters
- exibirDados()

Classe Livro

Atributos:

- titulo
- autor

Métodos:

- construtor
- getters e setters
- exibirDados()

Classe Emprestimo

Atributos:

- Usuario usuario
- Livro livro
- String dataEmprestimo
- String dataDevolucao

Métodos:

- realizarEmprestimo()
- devolverLivro()
- exibirEmprestimo()

Regras

O método realizarEmprestimo() deve exibir uma mensagem informando:

- o nome do usuário;
- o título do livro;
- a data do empréstimo.

Exercício 10:

Crie uma classe Gerente que estende de Funcionario, adicionando como atributo um vetor do tipo Funcionario chamado departamento e uma variável para controlar a quantidade de cadastrados. No construtor, além de receber os dados do gerente para passar à classe pai, receba também a capacidade máxima e inicialize o vetor. Implemente as funções: adicionar(), que recebe um funcionário por parâmetro e o adiciona no departamento; imprimirRelatorio(), que exibe na tela os dados de todos os funcionários alocados naquele departamento; atualizar(), que recebe o id de um funcionário junto com os novos dados (como nome e salário), localiza este no departamento e atualiza suas informações mantendo o id intacto; e remover(), que recebe o id de um funcionário, localiza este no departamento e remove o mesmo da equipe (atribuindo *null* à posição). Na classe principal, na função main, crie instâncias da classe Funcionario. Depois, crie um gerente definindo o tamanho do seu departamento e use a função de adicionar para colocar os empregados sob sua gestão. Por fim, execute e teste todas as funções do gerente.